Capture engine API

The capture engine turns a PCAP or PCAPNG file into a normalized capture: a set of TCP sessions, each with two directional streams, where every observed byte carries its provenance. It is the first stage of the pipeline; the protocol and UI stages consume its output and never touch the capture file themselves.

Module: `src.capture`. All public names are re-exported from `src.capture`.

Contents

- Quick start
- Typical scenarios
- Public API
- Reading provenance
- Diagnostics
- Limitations
- dpkt notes
- Performance

Quick start

Install the package with its development extras:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
```

Generate the synthetic fixtures and the corpus (both are deterministic):

```
python -m scripts.generate_fixture
python -m scripts.generate_corpus
```

Normalize a capture into the contract JSON and validate it:

```
python -m src.capture.cli --pcap tests/corpus/corpus_capture_01.pcapng \
    --out out/corpus_01.json --source-name captures/corpus_capture_01.pcapng
```

The same from Python:

```
from pathlib import Path
from src.capture import export_capture, normalize

capture = normalize(Path("tests/corpus/corpus_capture_01.pcapng"))
export_capture(
    capture.sessions,
    Path("out/corpus_01.json"),
    source_file="captures/corpus_capture_01.pcapng",
    capture_id=capture.capture_id,
    streams=capture.streams,
)
```

Typical scenarios

Parse a capture. Call `normalize(path)` to get sessions, both directional streams per session, and the diagnostics in one step. Use `read_capture(path)` alone when only the packets are needed, for example to count them.

Process a large capture. Add `--stream` to the CLI (or call `process_streaming`), which releases a session from memory once both sides have been closed by FIN or RST. A session that never closes is still held whole, so the peak never exceeds the in-memory pipeline. See Performance below.

Export to Wireshark. Add `--wireshark-pcap out.pcap` to write one packet per session direction carrying the reassembled bytes, so a stream can be inspected in Wireshark next to the original file. This is the Follow TCP Stream view without the interactive step.

Work with a defective capture. `read_capture` reads a damaged but recognizable file as far as it can and reports the rest as diagnostics instead of raising. Gaps and conflicting overlaps in a stream appear in `DirectionalStream.diagnostics`; parser trouble (truncation, fragments, mixed link types) appears in the diagnostics list passed to `read_capture`. The fixtures under `tests/fixtures/defects/` cover each case.

Public API

`read_capture(path, diagnostics=None, *, verify_checksums=False) -> Iterator[Packet]`

Reads `path` and yields every TCP/IPv4 packet in file order.

- `diagnostics` — optional list; notes about skipped link types, IPv6 frames, and non-TCP frames are appended to it. The iterator is unaffected.
- `verify_checksums` — when false (the default), checksums are not checked. When true, `Packet.checksum_valid` is set per packet; the caller decides what to do with a mismatch.

Raises `FileNotFoundError` if the file is missing and `ValueError` if the file is neither PCAP nor PCAPNG.

`Packet` (dataclass, frozen)

Field	Type	Meaning
`index`	`int`	0-based packet position in the file
`timestamp`	`float`	capture time, epoch seconds
`src_ip`, `dst_ip`	`str`	IPv4 addresses as text
`src_port`, `dst_port`	`int`	TCP ports
`seq`, `ack`	`int`	raw 32-bit sequence and acknowledgement numbers
`flags`	`int`	raw TCP flags
`payload`	`bytes`	TCP payload, empty for control segments
`is_truncated`	`bool`	the captured payload is shorter than the header promised
`checksum_valid`	`bool | None`	set only when checksums are verified

Convenience properties: `syn`, `fin`, `rst`, `ack_flag`, `payload_len`.

`Diagnostic` (dataclass, frozen)

Field	Type	Meaning
`type`	`str`	one of the types in the table under Diagnostics
`packet_index`	`int | None`	packet this note belongs to, when known
`offset`	`int | None`	stream offset, for stream-level notes
`length`	`int | None`	length in bytes, for stream-level notes
`detail`	`str`	human-readable explanation

`to_contract()` returns the dict written to the normalized JSON.

`build_sessions(packets) -> list[Session]`

Groups packets into TCP sessions. A session is a connection instance, not a 5-tuple: if the same address and port pair is reused later, that is a second session. Sessions appear in the order their first packet appears in the file, and `session_id` is `s1`, `s2`, and so on.

Roles are `client`/`server` only when a SYN was seen; otherwise both are `unknown`.

`Session` (dataclass)

Field	Type	Meaning
`session_id`	`str`	`s1`, `s2`, …
`endpoints`	`tuple[Endpoint, Endpoint]`	the two sides, A and B
`syn_seen`, `fin_seen`, `rst_seen`	`bool`	handshake and teardown flags
`isn_a`, `isn_b`	`int | None`	initial sequence numbers, when a SYN was seen
`first_ts`, `last_ts`	`float | None`	capture time bounds
`packet_indices`	`list[int]`	indices of the session's packets, in file order
`role_a`, `role_b`	`str`	`client`, `server`, or `unknown`

Methods: `endpoint_index(ip, port)`, `direction_of(packet)`, `to_contract()`.

`Direction` (enum)

`A_TO_B` and `B_TO_A`. `Direction.A_TO_B.opposite` gives the other one.

`Endpoint` (dataclass, frozen)

`ip: str`, `port: int`. `to_contract()` returns `{"ip": ..., "port": ...}`.

`reassemble(session, direction, packets, *, ignore_checksums=True) -> DirectionalStream`

Reassembles one direction of `session`. `packets` may contain the whole capture; packets that do not belong to the session or direction are ignored.

Algorithm: segments are ordered by sequence number, not by arrival time. Identical retransmissions add provenance but no bytes. Out-of-order segments are buffered and joined. A hole becomes a `gap` diagnostic, never zero bytes. A conflicting overlap keeps the first bytes and adds an `ambiguity` diagnostic. The 32-bit sequence number wrap is handled.

`DirectionalStream` (dataclass)

Field	Type	Meaning
`direction`	`Direction`	which way this stream flows
`bytes_`	`bytearray`	observed bytes only, holes omitted
`provenance`	`Provenance`	offset-to-packet map
`diagnostics`	`list[Diagnostic]`	gaps, ambiguities, checksum notes
`packet_indices`	`list[int]`	packets that carried this direction

Methods: `length`, `gaps()`, `ambiguities()`.

`Provenance` and `Range`

`Range` (dataclass, frozen): `offset`, `length`, `packet_index`, `seq`, `ts`. `end` is `offset + length`. `to_contract()` returns the dict for the JSON.

`Provenance`: `ranges: list[Range]`, plus:

- `add(rng)` — append a range.
- `lookup(offset) -> Range | None` — the range covering a stream offset, or `None` when the offset falls in a hole or past the end.
- `split(start, end) -> list[Range]` — ranges clipped to `[start, end)`, with `offset` and `seq` shifted so each piece still points at the right bytes.

`export_capture(sessions, out_path, *, source_file, capture_id, streams=None) -> None`

Writes the normalized capture as JSON, streaming session by session. `streams` maps a session id to its two directional streams.

`export_reassembled_pcap(sessions, out_path, *, streams=None, packets=None) -> int`

Writes one packet per session direction carrying the reassembled bytes, so a stream can be inspected in Wireshark next to the original capture. Returns the number of packets written. Header fields that a stream does not carry are fixed and documented in the module. The CLI exposes it as `--wireshark-pcap`.

Helpers

- `sha256_file(path) -> str` — `sha256:<hex>` of a file, used as `capture_id`.
- `capture_id_from_bytes(data) -> str` — same from bytes already in memory.
- `normalize(path) -> NormalizedCapture` — read, build sessions, and reassemble in one call. `normalize_bytes(data, source_file=...)` does the same from bytes. `normalize_packets(...)` reuses already-read packets.

`NormalizedCapture`: `capture_id`, `source_file`, `sessions`, `streams` (`dict[session_id, dict[Direction, DirectionalStream]]`), `diagnostics`, and `packets_read`.

Full example

```
from pathlib import Path

from src.capture import Direction, export_capture, normalize

capture = normalize(Path("tests/corpus/corpus_capture_01.pcapng"))

for session in capture.sessions:
    streams = capture.streams[session.session_id]
    a_to_b = streams[Direction.A_TO_B]
    print(session.session_id, len(a_to_b.bytes_), "bytes")

export_capture(
    capture.sessions,
    Path("out/corpus_01.json"),
    source_file="captures/corpus_capture_01.pcapng",
    capture_id=capture.capture_id,
    streams=capture.streams,
)
```

The same thing from the command line:

```
python -m src.capture.cli --pcap tests/corpus/corpus_capture_01.pcapng \
    --out out/corpus_01.json --source-name captures/corpus_capture_01.pcapng
```

Reading provenance

Given a stream offset, `lookup` returns the packet it came from:

```
from src.capture import Direction, normalize

capture = normalize(Path("capture.pcapng"))
stream = capture.streams["s1"][Direction.A_TO_B]

rng = stream.provenance.lookup(4)
if rng is None:
    print("offset 4 is a hole or past the end")
else:
    print(
        f"byte 4 came from packet {rng.packet_index} "
        f"at seq {rng.seq}, captured at {rng.ts}"
    )
```

To cover a byte range that spans packets or holes, use `split`; it returns one `Range` per contributing piece and never invents bytes:

```
for piece in stream.provenance.split(start, end):
    print(piece.offset, piece.length, piece.packet_index, piece.seq)
```

Diagnostics

type	Emitted by	Meaning
`gap`	reassembly	unobserved bytes; `offset` and `length` locate the hole
`ambiguity`	reassembly	the same bytes were seen with two different values; the first is kept and both are named in `detail`
`checksum_offload`	reassembly	a TCP checksum mismatch, only when checksums are verified
`ipv6_ignored`	parser	retained for files written before IPv6 was parsed; no longer emitted
`non_ip`	parser	a frame that is neither IPv4 nor IPv6
`non_tcp`	parser	an IP packet that is not TCP
`ip_fragment`	parser	retained for files written before fragment reassembly; no longer emitted
`ip_fragment_reassembled`	parser	a fragmented datagram was completed; `detail` names its fragment packets
`ip_fragment_incomplete`	parser	a fragmented datagram never completed; `offset` and `length` locate the hole
`ip_fragment_overlap`	parser	two fragments disagreed on the same bytes; IPv4 keeps the first arrival, IPv6 discards the datagram
`truncated_packet`	parser	the captured payload is shorter than the header declares
`truncated_frame`	parser	the frame was shorter than its headers claimed, or the capture header is damaged
`unsupported_linktype`	parser	the capture uses a link type other than Ethernet, loopback, or cooked, or mixes link types

Parser diagnostics land in `NormalizedCapture.diagnostics`. Stream diagnostics land in `DirectionalStream.diagnostics` and are written into the JSON with the stream. Truncation is also exposed per packet as `Packet.is_truncated`.

A `gap` diagnostic marks the point where the stream continues after a missing run: `offset` is the position in `bytes_` of the byte that follows the hole and `length` is how many bytes are missing before it. So the following byte also belongs to the stream, and `offset - length` is where the hole begins in the reconstructed ordering. `bytes_` itself never contains bytes for a hole.

Reading never stops at a defect. A capture that ends mid-block, a cut section header, mixed link types, fragmented datagrams, and mixed IP versions are all read as far as possible and reported; only a file that is not a capture at all raises `ValueError`. `tests/fixtures/defects/` holds one fixture per defect and `tests/capture/test_deformed_captures.py` covers each.

Fragment reassembly

A fragmented datagram is reassembled before it reaches the TCP layer, so a session that the network split becomes visible. Fragments are grouped by source, destination, identification, and protocol (IPv4) or by source, destination, and identification (IPv6); the fragment offset is in units of eight

bytes. A datagram is produced only once every byte from zero to its declared end is covered and the final fragment was seen.

- An overlap with different bytes keeps the first arrival for IPv4 and reports `ip_fragment_overlap`; for IPv6 the whole datagram is discarded, which is what RFC 5722 requires to remove the ambiguity.
- A datagram that never completes is reported as `ip_fragment_incomplete` with the hole's `offset` and `length`; its bytes are not guessed.
- A non-final IPv4 fragment whose length is not a multiple of eight is malformed and is not reassembled.
- Reassembly is complete for this purpose: fragments of one datagram may be separated by unrelated traffic and delivered out of order. Timers are not applied because the capture is read offline.

Limitations

- IPv6 is parsed. TCP sessions over IPv6 yield streams exactly like IPv4, and IPv6 TCP checksums are verified when `--verify-checksums` is set. IPv6 fragmentation is reassembled.
- IP fragmentation is reassembled for both versions. A datagram that never completes in the file is reported as incomplete rather than timed out.
- Encrypted traffic is not decrypted. TLS, and anything else above TCP, appears as opaque bytes.
- Only Ethernet, Linux loopback, and BSD cooked captures are understood; other link types are reported as `unsupported_linktype` and skipped.
- The whole capture is held in memory. See `docs/BENCHMARK.md` for the measured limit at the reference load profile.

dpkt notes

Three details cost time to find and are worth remembering when working on the parser:

- `dpkt.pcapng.Reader.datalink` is a method, while `dpkt.pcap.Reader.datalink` is an attribute. The parser calls it when callable.
- The push flag constant is `dpkt.tcp.TH_PUSH`, not `TH_PSH`.
- `dpkt.utils.inet_aton` does not exist; use `socket.inet_aton`.

Performance

Measured on the reference workload (the profile in `docs/BENCHMARK.md`: 95.66 MiB input, 250k packets, 1000 sessions, 122.44 MiB output JSON) on a 4 vCPU container:

Mode	Wall time	Peak RSS
In-memory (`normalize`)	11.56 s	354 MiB
Streaming (`--stream`)	11.22 s	224 MiB

Both are far inside the 5 minute and 4 GiB limits of the case. Streaming lowers the peak by releasing closed sessions; it does not change the output. Re-run `python -m scripts.run_benchmark --pcap .benchmark/profile.pcapng` to reproduce.